

```

1  (*
2
3      CS51 Lab 8
4      Functors
5      Part 1
6
7  (*
8
9      SOLUTION
10
11  (*=====
12  Readings:
13
14      This lab builds on material from Chapter 12.5 to the end of Chapter
15      12 of the textbook <http://book.cs51.io>, which should be read
16      before the lab session.
17
18  Objective:
19
20      This lab provides practice in the use of functors.
21
22      This part of the lab has been adapted from the functors chapter of
23      Real World OCaml <http://dev.realworldocaml.org/functors.html>.
24  =====*)
25
26  (*=====
27  Functors - Part 1.
28
29  For the first part of this lab, you will explore a realistic and
30  useful application of functors -- a library to support interval
31  computation.
32
33  Intervals come up in many different contexts. As a concrete example,
34  calendars need to associate events with time intervals (3-4pm or
35  11:30am-3:30pm). Intervals can be empty (like the interval starting at
36  4pm and ending at the previous 3pm; it contains no time at all). We
37  might want to know if a value is contained within an interval. (For
38  example, 4pm is not contained within 11:30am-3:30pm, but is contained
39  within 2-5pm.) We may want to know the intersection of two
40  intervals. (The intersection of 3-4pm and 11:30am-3:30pm is 3-3:30pm.)
41
42  Importantly, intervals can be defined over different data types. For
43  instance you may want to use intervals over floating point values or,
44  as in the example above, intervals of times. But regardless of the
45  type the intervals are over, you'll want some common basic operations:
46  constructing an interval, checking if an interval is empty, checking
47  if a point is contained within an interval, and intersecting two
48  intervals. (Of course, in an actual interval library, you'd want
49  several other operations as well, but this will be enough to get us
50  started.)
51
52  Let's examine how we can build a *generic* interval library using

```

```

53 functors.
54
55 An interval is defined by its endpoints. In order to operate on
56 intervals, we need to be able to manipulate the endpoints as well, in
57 particular by comparing two endpoints for how they are ordered. Thus,
58 to define intervals over a certain kind of endpoints, we'll need the
59 type of the endpoints and a way to compare two endpoints.
60
61 The `ORDERED_TYPE` signature from the textbook (Section 12.3) will
62 serve us nicely here. It uses the OCaml convention for comparison
63 functions used by the `Stdlib.compare` function:
64
65     compare x y < 0      (* x < y *)
66     compare x y = 0      (* x = y *)
67     compare x y > 0      (* x > y *)
68 *)
69
70 module type ORDERED_TYPE =
71   sig
72     type t
73     val compare : t -> t -> int
74   end ;;
75
76   (*****
77    A digression: It can be quite confusing which `compare` function
78    we are referring to in the code in this lab, between the `compare`
79    functions in implementations of the `ORDERED_TYPE` signature and
80    the `compare` function that is always available by virtue of the
81    opening of the `Stdlib` module. To regain some clarity, we'll
82    "turn off" the unqualified `Stdlib` function. *)
83
84   let compare _ = failwith "Use `Stdlib.compare` instead" ;;
85
86   (* You can still use that function, but you'll need to qualify it
87   with its module name as `Stdlib.compare`.
88   *****)
89
90   (*.....
91   Exercise 1A: Complete the following functor for making interval modules.
92
93   We represent an interval with a variant type, which is either `Empty`
94   or `Interval (x, y)`, where `x` and `y` are the bounds of the
95   interval, and are themselves contained within the interval. The
96   representation has an invariant that `x` is always less than or equal
97   to `y` (where the notion of "less than or equal" is given by the
98   `compare` function provided by the `ORDERED_TYPE` that the intervals
99   are built from).
100
101   The functor definition starts out as
102
103       module MakeInterval (Endpoint : ORDERED_TYPE) =
104         struct
105           ...
106         end

```

```

107
108 Here, the argument to the `MakeInterval` functor is a module, given
109 the name `Endpoint` and constrained to satisfy the `ORDERED_TYPE`
110 module signature. Thus, we know that `Endpoint` will provide both a
111 type (`Endpoint.t`) and a comparison function over that type
112 (`Endpoint.compare`). We can and will use those in defining the module
113 being defined by the functor.
114
115 Now, complete the functor definition below.
116 .....*)
117
118 module MakeInterval (Endpoint : ORDERED_TYPE) =
119   struct
120     type interval =
121       | Interval of Endpoint.t * Endpoint.t
122       | Empty
123
124     (* create low high -- Returns a new interval covering `low` to
125      * `high` inclusive. If `low` is greater than `high`, then the
126      * interval is empty. *)
127     let create (low : Endpoint.t) (high : Endpoint.t) : interval =
128       if Endpoint.compare low high > 0 then Empty
129       else Interval (low, high)
130
131     (* is_empty intvl -- Returns true if and only if `intvl` is
132      * empty *)
133     let is_empty (intvl : interval) : bool =
134       match intvl with
135       | Empty -> true
136       | Interval _ -> false
137
138     (* contains intvl x -- Returns true if and only if the value `x`
139      * is contained within `intvl` *)
140     let contains (intvl : interval) (x : Endpoint.t) : bool =
141       match intvl with
142       | Empty -> false
143       | Interval (low, high) ->
144         Endpoint.compare x low >= 0
145         && Endpoint.compare x high <= 0
146
147     (* intersect intvl1 intvl2 -- Returns the intersection of `intvl1`
148      * and `intvl2` *)
149     let intersect (intvl1 : interval) (intvl2 : interval) : interval =
150       let ordered x y =
151         if Endpoint.compare x y <= 0 then x, y
152         else y, x in
153       match intvl1, intvl2 with
154       | Empty, _
155       | _, Empty -> Empty
156       | Interval (low1, high1), Interval (low2, high2) ->
157         let (_, low), (high, _)
158           = ordered low1 low2, ordered high1 high2 in
159         create low high

```

```

160     end ;;
161
162 (*.....
163 Exercise 1B: Using the completed functor above, instantiate an
164 *integer* interval module.
165 .....*)
166
167 module IntInterval =
168   MakeInterval (struct
169     type t = int
170     let compare = Stdlib.compare
171   end) ;;
172
173 (*.....
174 Exercise 1C: Using your newly created integer interval module, create
175 two non-empty intervals named `intvl1` and `intvl2` that have some
176 overlap, and calculate their intersection as `intvl1_intersect_intvl2`.
177 .....*)
178
179 let intvl1 = IntInterval.create 3 8 ;;
180 let intvl2 = IntInterval.create 4 10 ;;
181 let intvl1_intersect_intvl2 = IntInterval.intersect intvl1 intvl2 ;;
182
183 (* There's currently a problem with the `MakeInterval` functor. It's
184 not abstract enough. Notably we are working with an invariant that a
185 valid non-empty interval has an upper bound that is greater than or
186 equal to its lower bound. However, this is only enforced by the
187 `create` function, and as it turns out, we can actually bypass the
188 `create` function due to our lack of an abstraction barrier.
189
190 This expression returns `true`, as expected.
191
192   IntInterval.is_empty (IntInterval.create 4 3) ;;
193
194 This, however, returns `false`. Yikes.
195
196   IntInterval.is_empty (IntInterval.Interval (4, 3)) ;;
197
198 **Make sure you understand why this is a problem. If you don't see the
199 issue, call over a staff member to discuss.**
200
201 To make our functor more abstract, we need to restrict the output of
202 `MakeInterval` to an interface that prevents users from directly
203 creating interval implementations themselves without using the
204 `create` function.
205
206 .....
207 Exercise 2A: Complete the following interface for an interval
208 module. Note in particular that we should add a new type `endpoint` to
209 give us a way of abstractly referring to the type for the endpoints of
210 an interval.
211 .....*)
212
213 module type INTERVAL =

```

```

214 sig
215   type interval
216   type endpoint
217   val create : endpoint -> endpoint -> interval
218   val is_empty : interval -> bool
219   val contains : interval -> endpoint -> bool
220   val intersect : interval -> interval -> interval
221 end ;;
222
223 (*.....
224 Exercise 2B: Augment the `MakeSafeInterval` functor using the code you
225 wrote for `MakeInterval` as a starting point, such that it returns a
226 module restricted to the `INTERVAL` signature. (Much of the
227 implementation can be copied from `MakeInterval` above.) **Don't
228 forget to specify the module type.**
229 .....*)
230
231 (* Note the specification of the module type here -----+
232                                     v                                     *)
233 module MakeSafeInterval (Endpoint : ORDERED_TYPE) : INTERVAL =
234
235   (* The rest of the implementation is just as before! *)
236   struct
237     type endpoint = Endpoint.t
238     type interval =
239       | Interval of endpoint * endpoint
240       | Empty
241
242     (* create low high -- Returns a new interval covering `low` to
243        `high` inclusive. If `low` > `high`, then the interval is
244        empty. *)
245     let create (low : endpoint) (high : endpoint) : interval =
246       if Endpoint.compare low high > 0 then Empty
247       else Interval (low, high)
248
249     (* is_empty intvl -- Returns true if and only if the `intvl` is
250        empty *)
251     let is_empty (intvl : interval) : bool =
252       match intvl with
253       | Empty -> true
254       | Interval _ -> false
255
256     (* contains intvl x -- Returns true if and only if the value `x`
257        is contained within `intvl` *)
258     let contains (intvl : interval) (x : Endpoint.t) : bool =
259       match intvl with
260       | Empty -> false
261       | Interval (low, high) ->
262         Endpoint.compare x low >= 0
263         && Endpoint.compare x high <= 0
264
265     (* intersect intvl1 intvl2 -- Returns the intersection of the two
266        input intervals *)
267     let intersect (intvl1 : interval) (intvl2 : interval) : interval =

```

```

268     let ordered x y = if Endpoint.compare x y <= 0 then x, y else y, x in
269     match intv11, intv12 with
270     | Empty, _
271     | _, Empty -> Empty
272     | Interval (low1, high1), Interval (low2, high2) ->
273         let (_, low), (high, _) = ordered low1 low2, ordered high1 high2 in
274         create low high
275     end
276 ;;
277
278 (* We have successfully made our returned module abstract, but believe
279 it or not, it is now too abstract. In fact, we have not exposed the
280 type of endpoints to the user, meaning we cannot even create intervals
281 now. The abstraction barrier is too strong. To demonstrate the problem
282 ...
283
284 .....
285 Exercise 2C: Create an `IntSafeInterval` module using the new
286 `MakeSafeInterval` functor.
287 .....*)
288
289 module IntSafeInterval =
290     MakeSafeInterval
291     (struct
292         type t = int
293         let compare = Stdlib.compare
294     end) ;;
295
296 (* Now, try evaluating the following expression in the REPL:
297
298     IntSafeInterval.create 2 3 ;;
299
300 A type error will appear:
301
302     Error: This expression has type int but an expression was expected of type
303           IntInterval.endpoint
304
305 To make the interface slightly less abstract, we can make use of a
306 sharing constraint, which informs the compiler that a given type
307 *within* the implementation is equal to some other type from *outside*
308 the implementation. In this case, we want to inform the compiler the
309 type of our endpoint is an `int`, and more generally that the type of
310 our endpoint is `Endpoint.t`, where `Endpoint` was the `ORDERED_TYPE`
311 module that is the argument of the functor. We can do so with the
312 following syntax:
313
314     <Module_type> with type <type> = <type'>
315
316 For instance, we can create int interval and float interval interfaces
317 that reveal the type of endpoints as follows: *)
318
319 module type INT_INTERVAL =
320     INTERVAL with type endpoint = int ;;
321

```

```

322 module type FLOAT_INTERVAL =
323   INTERVAL with type endpoint = float ;;
324
325 (* Modules that satisfy these interfaces will allow users to actually
326 construct intervals of the desired types.
327
328 While sharing constraints solve the abstraction issue discussed
329 earlier, they now present a new problem. They will result in code
330 duplication in implementation. The solution to this is to use sharing
331 constraints in the MakeInterval functor, exposing that the type of an
332 endpoint is equal to Endpoint.t. The functor can then be used to
333 create interval modules of various types without duplicating
334 code. That's what we'll do next. *)
335
336 (*.....
337 Exercise 3A: Define a new functor `MakeBestInterval`. It should take an
338 ORDERED_TYPE module for the endpoints of the intervals, and return a
339 module satisfying INTERVAL *with appropriate sharing constraints
340 to allow the creation of generic interval modules*.
341
342 If you do this correctly, your code for `MakeBestInterval` should be
343 almost identical to your code for `MakeSafeInterval` with the
344 exception of any needed sharing constraints.
345 .....*)
346
347 module MakeBestInterval (Endpoint : ORDERED_TYPE)
348   : (INTERVAL with type endpoint = Endpoint.t) =
349     (* ^
350      |---- Note the addition of the sharing constraint.
351      Literally everything else stays the same. *)
352   struct
353     type endpoint = Endpoint.t
354     type interval =
355       | Interval of endpoint * endpoint
356       | Empty
357
358     (* create low high -- Returns a new interval covering `low` to
359      `high` inclusive. If `low` > `high`, then the interval is
360      empty. *)
361     let create (low : endpoint) (high : endpoint) : interval =
362       if Endpoint.compare low high > 0 then Empty
363       else Interval (low, high)
364
365     (* is_empty intvl -- Returns true if and only if the `intvl` is
366      empty *)
367     let is_empty (intvl : interval) : bool =
368       match intvl with
369       | Empty -> true
370       | Interval _ -> false
371
372     (* contains intvl x -- Returns true if and only if the value `x`
373      is contained within `intvl` *)
374     let contains (intvl : interval) (x : Endpoint.t) : bool =
375       match intvl with

```

```

376 | Empty -> false
377 | Interval (low, high) ->
378   Endpoint.compare x low >= 0
379   && Endpoint.compare x high <= 0
380
381 (* intersect intvl1 intvl2 -- Returns the intersection of the two
382    input intervals *)
383 let intersect (intvl1 : interval) (intvl2 : interval) : interval =
384   let ordered x y = if Endpoint.compare x y <= 0 then x, y else y, x in
385   match intvl1, intvl2 with
386   | Empty, _
387   | _, Empty -> Empty
388   | Interval (low1, high1), Interval (low2, high2) ->
389     let (_, low), (high, _) = ordered low1 low2, ordered high1 high2 in
390     create low high
391 end
392 ;;
393
394 (* We now have a fully functioning functor that can create interval
395    modules of whatever type we want, with the appropriate abstraction
396    level.
397
398    .....
399    Exercise 3B: Use the `MakeBestInterval` functor to create a new int
400    interval module called `IntBestInterval`, and test that it works as
401    expected.
402
403    You may for instance want to try the problematic lines from Exercise 1C.
404
405    The following expression should still return `true`, as expected:
406
407        IntBestInterval.is_empty (IntBestInterval.create 4 3) ;;
408
409    and this expression should no longer return `false`. What does it
410    return instead?
411
412        IntBestInterval.is_empty (IntBestInterval.Interval (4, 3)) ;;
413    .....*)
414
415 module IntBestInterval =
416   MakeBestInterval
417   (struct
418     type t = int
419     let compare = Stdlib.compare
420   end) ;;

```